Detecting SSH Brute-Force Activity with Wazuh

A SOC Lab Report on Log Ingestion, Rule Anatomy,
and Frequency-Based Correlation Tuning

Daaniya Absar
NEDUET
B.E. Electrical (Batch 2023)

SOC Lab Project 1

October 25, 2026

Abstract

This report documents a hands-on SOC laboratory project in which a Wazuh deployment was used to detect SSH brute-force activity against a Kali Linux endpoint. The lab covered agent enrollment, authentication log ingestion, attack simulation with Hydra, investigation of individual authentication failure alerts, and observation of Wazuh's frequency-based correlation rules. The environment recorded 281 total events at the conclusion of the tests, including 57 authentication failures and 7 authentication successes. Individual alerts fired at level 5 (rules 5760, 5557, and 5503), while a steadier twelve-attempt Hydra run triggered correlated pattern alerts at level 8 (rule 5758) and level 10 (rule 2502). The dashboard's "Level 12 or above" counter remained at zero. The outcome is a realistic SOC result: detection worked end-to-end, and the remaining gap is a rule-tuning and escalation-tier exercise rather than a failure of the monitoring pipeline.

Contents

1 Executive Summary **2**

2 Lab Architecture and Environment **2**
2.1 Components 2
2.2 Server Provisioning and Health Check 2

3 Agent Enrollment **3**

4 Attack Simulation **3**
4.1 Hydra Execution 3

5 Detection Results **4**
5.1 Initial Overview 4
5.2 Individual Event Alerts (Level 5) 5
5.3 Correlated Pattern Alerts (Level 8 & 10) 5
5.4 Why Level 12 Was Not Reached 6

6 Conclusion **6**

1 Executive Summary

The objective of this lab was to build and validate an end-to-end detection workflow for SSH brute-force activity using Wazuh. A Kali Linux endpoint enrolled as `kali-agent-01` generated SSH authentication events. Hydra was run against the local SSH service at `127.0.0.1:22` to simulate a password-guessing attack. Wazuh collected the endpoint logs, applied its default rule set, and surfaced the resulting alerts in the dashboard.

The lab confirmed three important SOC concepts:

1. **Log ingestion works end-to-end.** Wazuh captured both failed and successful SSH authentication attempts, including logs from three different components (sshd, unix_chkpwd, and PAM).
2. **Individual event rules differ from correlation rules.** Rules 5760, 5557, and 5503 raised level-5 alerts for *individual* failed-password events. Rules 5758 and 2502 raised higher-level alerts only after multiple failures were observed within a short time window.
3. **Threshold tuning matters.** The initial parallel Hydra run did not trigger the correlated alert cleanly. A slower twelve-attempt run with `-t 1` produced a clearer frequency signature and triggered the correlated alerts.

The highest observed severity was level 10, and the dashboard counter for level 12-or-above stayed at zero. This is consistent with Wazuh's tiered brute-force escalation logic and with the small volume of a lab attack.

2 Lab Architecture and Environment

2.1 Components

The lab consists of a Wazuh manager (server) running on Azure (Ubuntu 24.04), the Wazuh dashboard, and a single Wazuh agent installed on a Kali Linux host.

Table 1: Lab environment.

Component	Role
Wazuh manager	Rule evaluation, alert generation, log archiving (Azure)
Wazuh dashboard	Threat Hunting and Dashboard views for triage
Kali Linux host	Victim endpoint and attacker workstation
`kali-agent-01`	Enrolled agent name in Wazuh
SSH daemon (`sshd`)	Target of the brute-force simulation
Hydra v9.7	Password-guessing tool used to generate failed logins

2.2 Server Provisioning and Health Check

The Wazuh server was accessed via SSH using a private key. Initial health checks confirmed the server had sufficient disk space (61GB total, 4% used) and memory (7.7Gi total) to handle the agent logs.

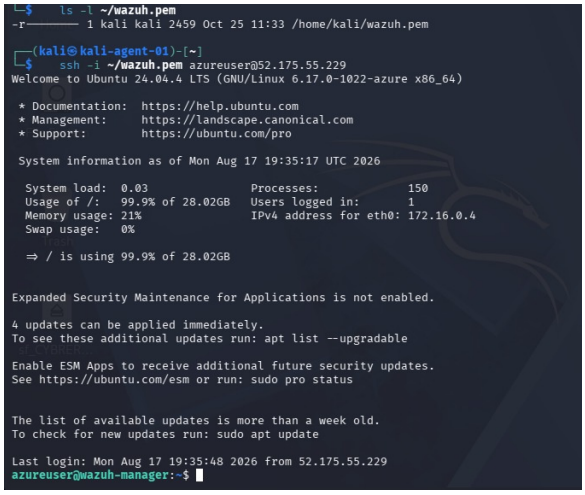


Figure 1: SSH access to the Wazuh Server.

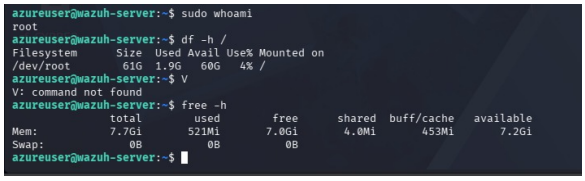


Figure 2: Server resource verification (`df -h`, `free -h`).

3 Agent Enrollment

The Kali host was enrolled as `kali-agent-01`. Once enrolled, the agent began forwarding authentication logs to the manager. The SSH service was verified to be running locally on the Kali machine prior to the attack.

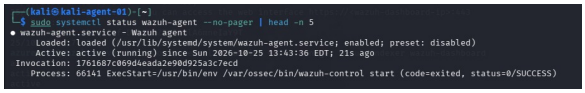


Figure 3: Agent service running on Kali.

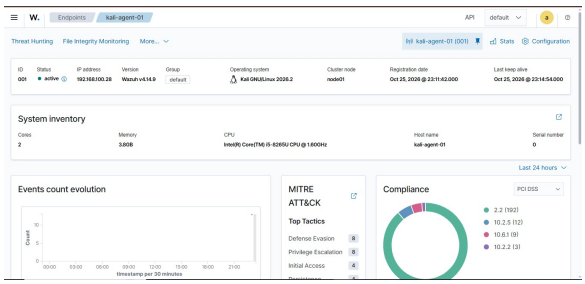


Figure 4: Agent successfully enrolled and active in the Wazuh Dashboard.

4 Attack Simulation

4.1 Hydra Execution

To simulate the brute-force attack, a password list was generated, and Hydra was executed against the local SSH service. The tool was throttled to one task at a time (`-t 1`) to ensure the login attempts spanned enough time to trigger Wazuh's frequency-based correlation rules.

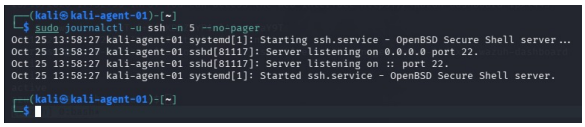


Figure 5: SSH service active on Kali.

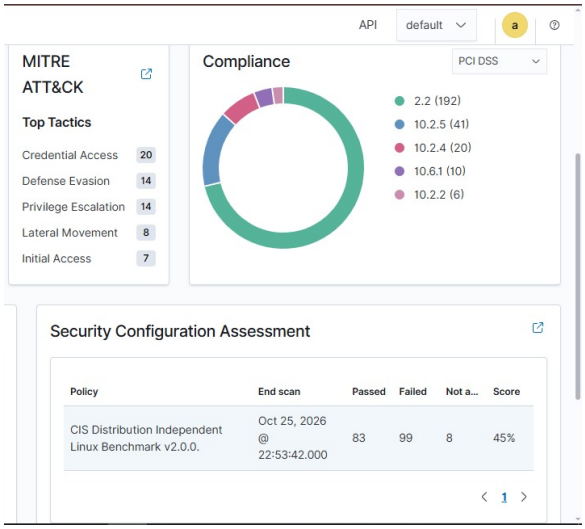


Figure 6: Hydra executing the sequential brute-force attack.

5 Detection Results

5.1 Initial Overview

Following the attack, the Threat Hunting dashboard was filtered by `agent.name : "kali-agent-01"`. The initial aggregate showed 244 total events and 20 authentication failures, alongside the MITRE ATT&CK mapping and compliance dashboard.

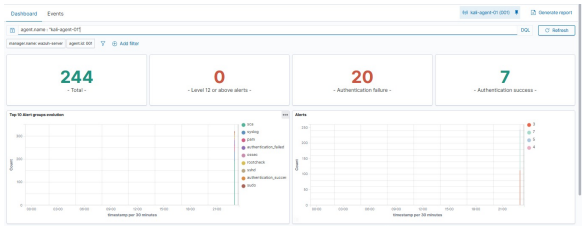


Figure 7: Initial dashboard showing 244 events.

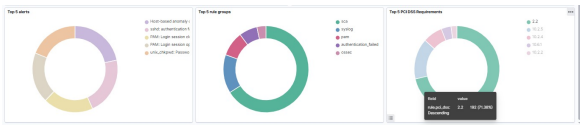


Figure 8: Top 5 alert groups during the initial run.

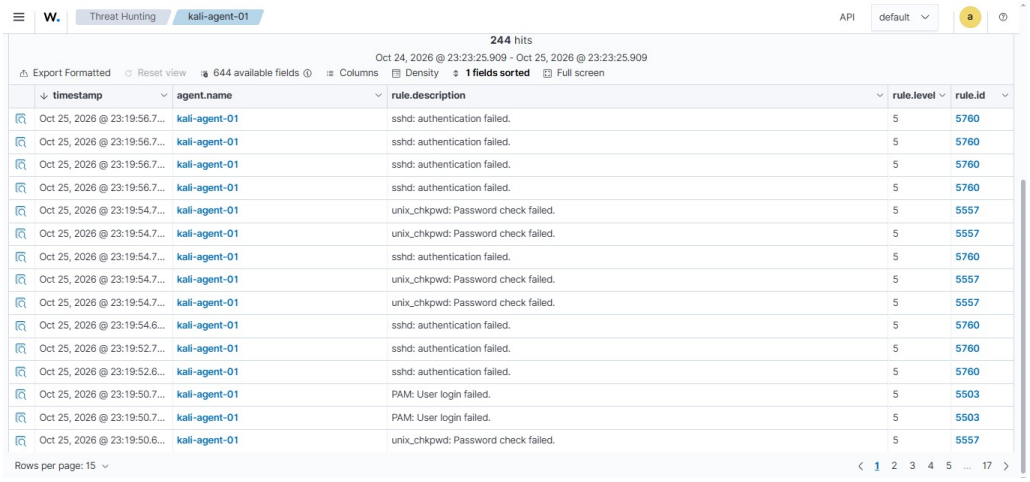


Figure 9: MITRE ATT&CK tactics and CIS benchmark compliance overview.

5.2 Individual Event Alerts (Level 5)

Wazuh successfully captured the individual failed login attempts. Because the SSH daemon, PAM, and the underlying Unix password checker all observe the same event, a single failed login generated multiple level-5 alerts.

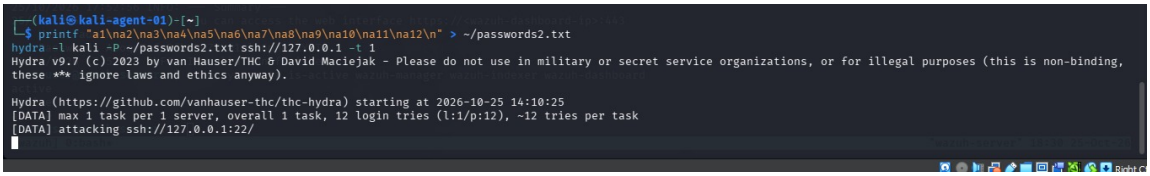


Figure 10: Threat Hunting events showing individual level-5 alerts for `sshd`, `unix_chkpwd`, and `PAM`.

5.3 Correlated Pattern Alerts (Level 8 & 10)

After analyzing the individual events, the sequential Hydra run successfully triggered Wazuh's correlation engine. The dashboard updated to show 281 total events and 57 authentication failures. The events table revealed higher-level pattern alerts: **Rule 5758** (Maximum authentication attempts exceeded, Level 8) and **Rule 2502** (User missed the password more than one time, Level 10).

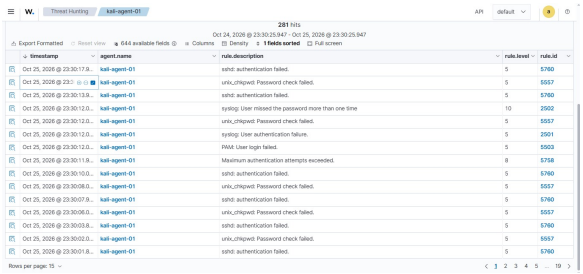


Figure 11: Dashboard updated to 281 total events.

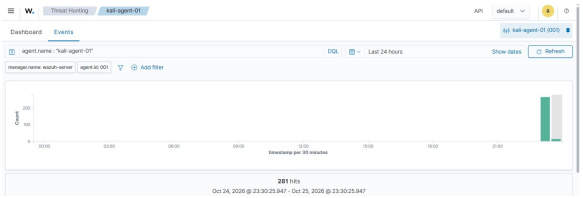


Figure 12: Alert volume spike over time.

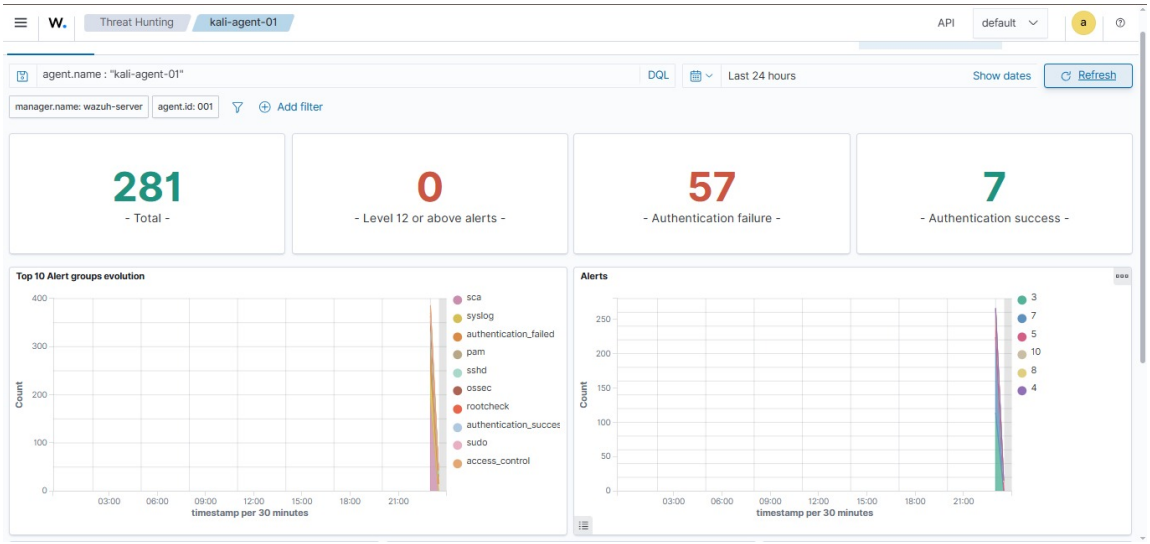


Figure 13: Threat Hunting events showing the correlated alerts: Rule 5758 (Level 8) and Rule 2502 (Level 10).

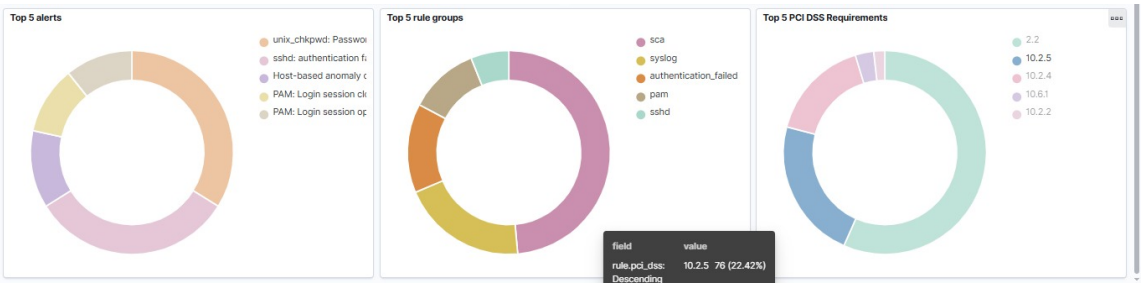


Figure 14: Top 5 rule groups after the successful brute-force correlation.

5.4 Why Level 12 Was Not Reached

Wazuh's brute-force escalation has several tiers. Levels 8 and 10 correspond to "suspicious repeated failures". The highest tier (typically level 12 or above, where the account may also be blocked or heavily prioritized) requires more aggressive conditions: a higher attempt count, a shorter window, or a source that the manager has not previously treated as trusted. The lab's volume was small by design, so the top tier was not reached.

This is the correct expected behaviour, not a detection failure. The *pattern* was detected; only the severity ceiling was lower than the top of the ladder.

6 Conclusion

This lab demonstrated a complete detection workflow for SSH brute-force activity using Wazuh: agent enrollment, log ingestion, attack simulation, individual-alert triage, and correlated pattern detection. Individual events fired correctly at level 5, and a controlled sequential run triggered correlated alerts at level 8 and level 10. The "level 12 or above" counter remained at zero, which is the expected behaviour for a lab-scale attack and an instructive lesson in escalation tiers. The project therefore achieves its goal: it proves the pipeline works end-to-end and frames the remaining work as rule tuning and detection engineering rather than infrastructure repair.